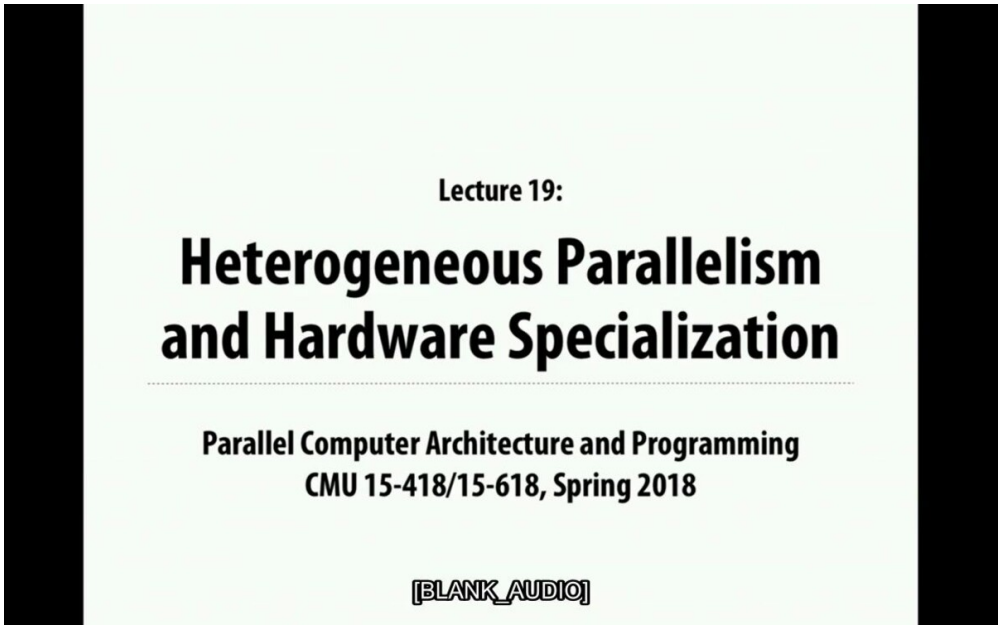
课程笔记

异构并行：功耗约束下的胖核、瘦核与专用加速

CMU 15-418/618 并行计算机架构与编程 · Lecture 23（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 20 分 12 秒
视频链接：`https://www.bilibili.com/video/BV1R8411Q7cB/`

目录

1 功耗：改变设计思路的主导约束 2
1.1 从手机到超算的共同约束 . . . 2
1.2 已有硬件大多被闲置 . . . 2
1.3 本章小结 . . . 3

2 固定面积下的胖核与瘦核 3
2.1 一个选择题 . . . 3
2.2 把 Amdahl 定律改写成资源形式 . . . 4
2.3 曲线怎么读 . . . 5
2.4 本章小结 . . . 6

3 异构方案：一个胖核加若干瘦核 7
3.1 模型设定 . . . 7
3.2 曲线更平缓 . . . 8
3.3 本章小结 . . . 9

4 现实约束：为什么不能照模型定制 9
4.1 三条限制 . . . 9
4.2 本章小结 . . . 10

5 真实系统里的异构 10
5.1 移动 SoC . . . 10
5.2 片上集成的关键优势 . . . 11
5.3 再加一块独立 GPU . . . 12
5.4 本章小结 . . . 13

6 超算与专用化 13
6.1 本章小结 . . . 13

7 超算排名与专用化 13
7.1 单一基准的问题 . . . 13
7.2 专用化：走向硬件 . . . 15
7.3 本章小结 . . . 15

8 总结与延伸 16

1　功耗：改变设计思路的主导约束

这一讲讨论一个有意思的话题：异构计算环境有什么好处，以及从纯软件走向更接近硬件（甚至自己做硬件设计）能带来什么。

1.1　从手机到超算的共同约束

讲者开场给出一个贯穿性观察：从手机到超级计算机，都在使用非常混合的计算引擎，而且原因各不相同。

但真正主导当代计算机设计的是功耗。超算机房供电不能超过 10 兆瓦；手机若耗电太多，既会耗尽电池，也会热到无法握在手里或放在口袋里。

因此无论最小的还是最大的计算机，功耗都是那个问题。过去十年里，这一点改变了很多关于”什么该做、什么不该做”的思考。

功耗约束把”性能”重新定义为”每瓦性能”

在供电受限的前提下，增加算力的唯一途径是提高每瓦效率，而不是简单地堆更多晶体管。这解释了为什么专用加速器有吸引力：它在特定任务上能效远高于通用核，因此同样功耗预算下能完成更多工作。

1.2　已有硬件大多被闲置

讲者提醒一个事实：我们在单核系统上已经见过极大的性能差异，取决于你愿意花多大力气。

作业一里，如果你使劲用上 SIMD 向量单元和多线程，就能在不换任何硬件的情况下让应用大幅加速。

”买更多硬件”与”用更好现有硬件”是两条不同的路

前者需要更换系统，后者只需改代码与算法。第 8 讲与第 15 讲的例子都说明后者的收益可以很大且成本更低。讲者在此提出这一点，是为了说明异构讨论之前还有一层更基础的问题：现有资源是否已被充分利用。

他的判断是：典型机器拥有的硬件远多于多数程序实际使用的部分。特别是整个 SIMD 向量单元，除了那些有人花大力气优化过的库之外，基本处于闲置状态。

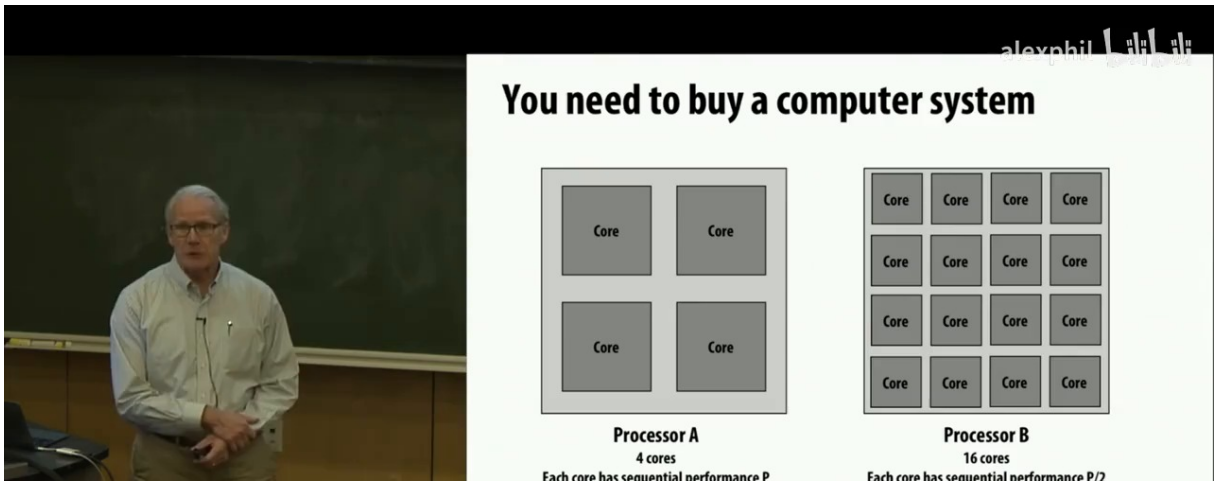


图 1: 你需要买一台计算机系统：固定面积下如何分配核[1]

1.3 本章小结

功耗是从手机到超算的共同主导约束，它把性能问题重新定义为每瓦性能问题。同时，现有硬件的相当部分（尤其 SIMD 单元）在多数程序中被闲置。因此”异构”与”更充分利用已有硬件”是同一件事的两面。

下一章用一个固定面积的简化模型，把胖核与瘦核的取舍算清楚。

2 固定面积下的胖核与瘦核

2.1 一个选择题

讲者先回顾一道考题：如果可以在 k 个性能为 1 的处理器与 1 个性能为 k 的处理器之间选择，你选哪个？

Amdahl 定律使”等价交换”不成立

k 个性能为 1 的核，理论峰值是 k；1 个性能为 k 的核，峰值也是 k。但前者受串行部分与并行开销拖累，实际只能达到远低于 k 的加速比，而后者没有这些损失。因此两者峰值相同但实际收益不同。

答案应当是后者，因为你几乎拿不到理想加速比，所以这不是等价交换。

[1] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

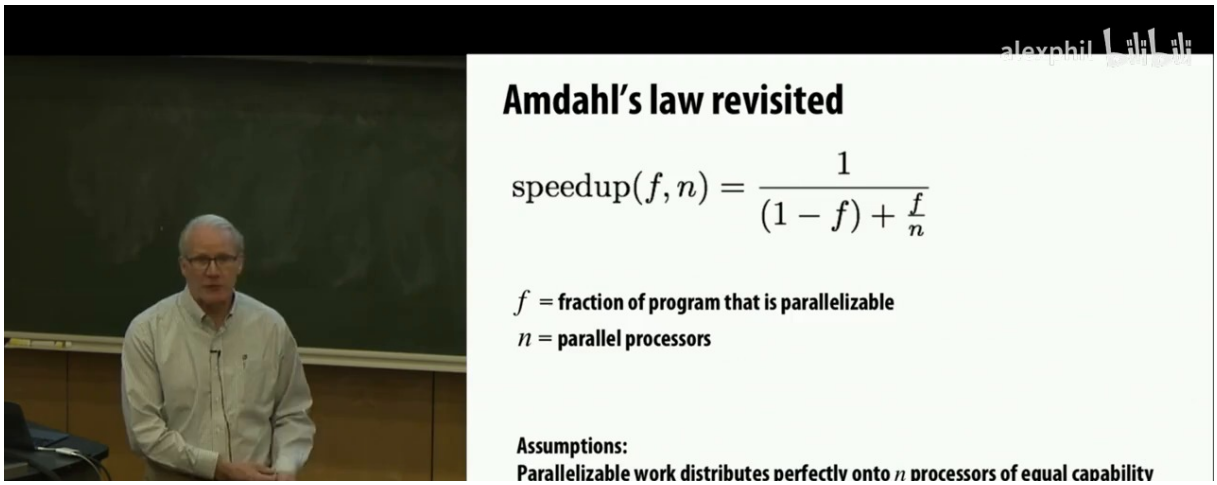


图 2: 重访 Amdahl 定律：加速受未并行部分限制[2]

但他接着把这个问题做得更细：设想在固定芯片面积下，面积并不均等划分。假设胖核的性能恰好是瘦核的两倍，那么右侧系统的原始算力可达八倍。

2.2 把 Amdahl 定律改写成资源形式

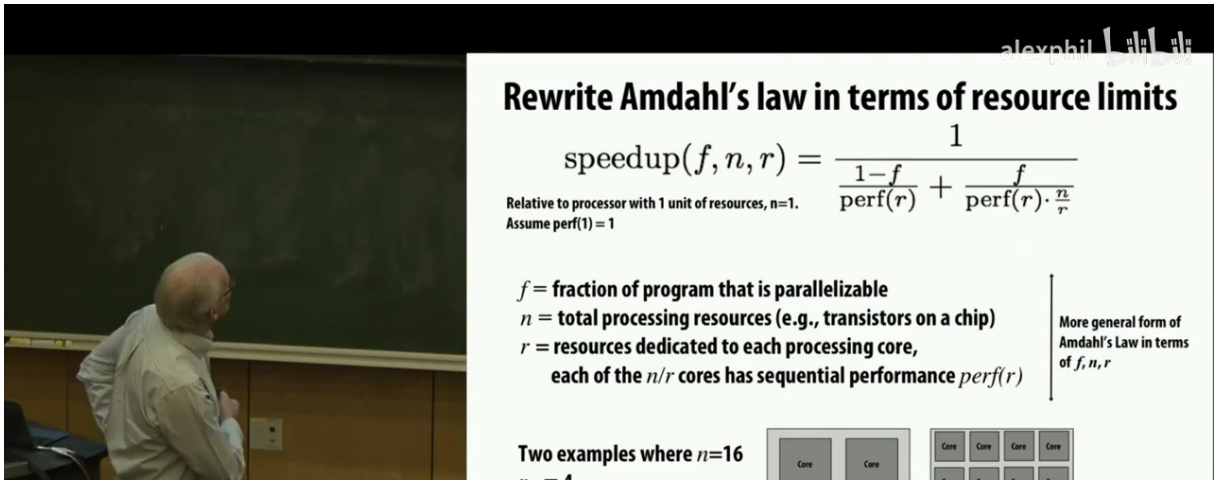


图 3: 用资源限制改写 Amdahl 定律[3]

讲者把 Amdahl 定律重新表述。经典形式是：

$$S(f,n) = \frac{1}{(1-f)+\frac{f}{n}}$$

- f：程序中可并行部分的比例；
- n：并行处理器数；
- 当 $n \to \infty$ 时，S 的上界为 $1/(1-f)$。

[2] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。
[3] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

而资源形式需要引入核大小。假设总资源为 N，每核大小为 r，则核数为 N/r，每核性能为 $\mathrm{perf}(r)$：

$$S(f, N, r) = \frac{1}{\dfrac{1-f}{\mathrm{perf}(r)} + \dfrac{f \cdot r}{N \cdot \mathrm{perf}(r)}}$$

- $\mathrm{perf}(r)$：单核性能随其分得资源 r 的函数；
- N/r：核的数量；
- 关键点是 $\mathrm{perf}(r)$ 呈收益递减，而非线性增长。

收益递减是这套模型的唯一关键假设

讲者说你可以选任何喜欢的函数，但收益递减是普遍性质。他取的示例是：

$$\mathrm{perf}(r) = \sqrt{r}$$

- r：单个核分得的资源份数；
- $r = 1$ 时性能为 1，作为归一化基准；
- $r = 4$ 时性能为 2，$r = 16$ 时性能为 4。

面积翻四倍而性能只翻两倍，这就是”递减”的具体含义。他取 $\mathrm{perf}(r) = \sqrt{r}$ 作示例：$r = 4$ 时性能只有 2 倍，$r = 16$ 时只有 4 倍。这个选择的具体形式不重要，重要的是它必须递减——只要递减，后面的结论就成立。

2.3 曲线怎么读

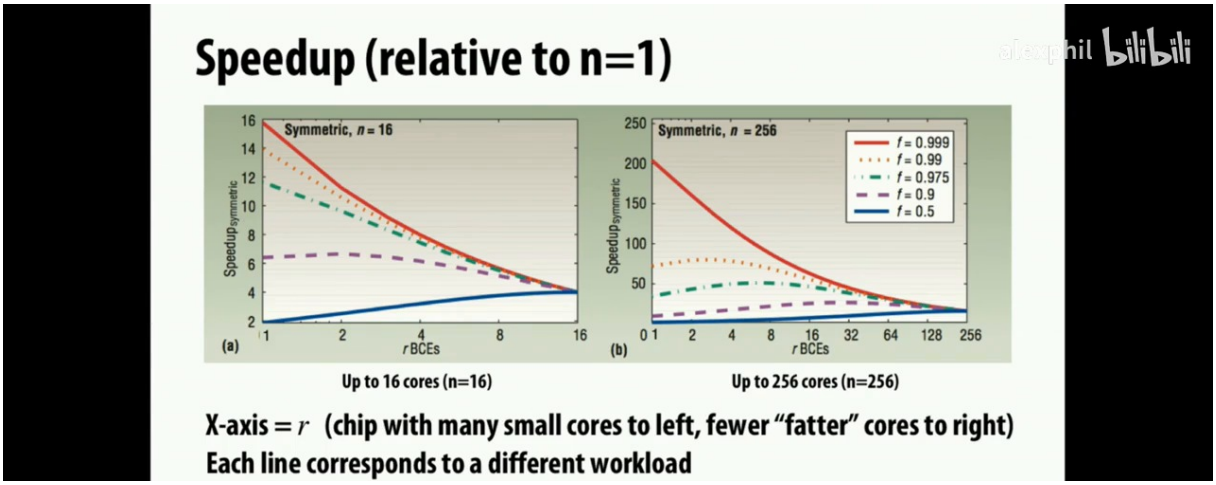


图 4: 加速比曲线（相对单核）：对称 16 核情形[4]

讲者画出”对称情形”的曲线族，也就是所有核都相同、每核分得 r 份资源。

横轴是核大小，纵轴是相对”一个极小的单核”的加速比。问题是：把面积放大 16 倍，究竟能得到多少实际收益？

[4]视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

结果取决于并行比例：

- 若几乎完美并行（串行只占 0.01%），那么把芯片切成尽可能多的小核就是最优，几乎能拿到完美的 16 倍；
- 但只要串行比例升到 1%，就开始出现明显损失；
- 到 10% 时，最多只能拿到约 6 倍多；
- 若只有一半可并行，加速比只有 2 倍左右。

收敛点揭示了模型的边界

所有曲线收敛到同一个点，说明无论并行比例如何，把整块面积做成一个核都是同一个结果。这意味着模型的差异全部体现在”中间区域”：核不太大也不太小的配置。因此设计决策的关键区间正是中间地带，而两端是退化情形。

所有曲线都收敛到同一个点：单核占满整个芯片，此时 $r = 16$，性能为 4 倍（按平方根模型）。

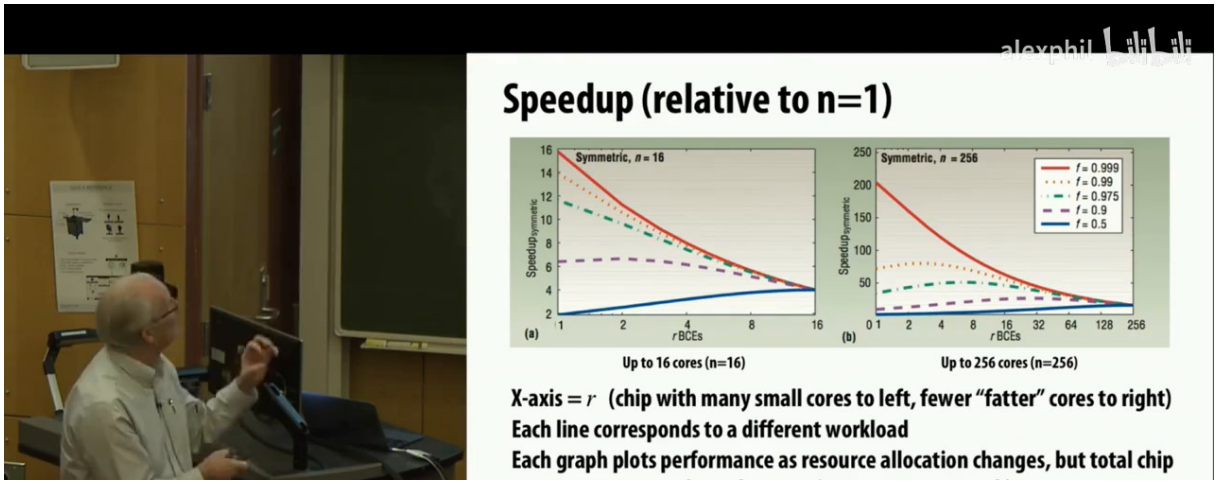


图 5: 面积放大 256 倍的加速比曲线：Amdahl 效应更严峻[5]

若把面积放大到 256 倍，原理相同但更极端。即便九成九可以并行，加速比也只有 70 倍左右，尽管芯片面积大了 256 倍，这个数字远低于直觉预期。

同构设计在两端都不讨好

若把面积都做成胖核，收益因递减而有限；若都做成瘦核，则串行部分会成为硬瓶颈。因此同构方案在”并行度不高”与”并行度极高”两种极端下都不是最优。这正是异构设计的出发点。

2.4 本章小结

固定面积下，胖核与瘦核的取舍由串行比例决定：几乎全并行时小核最优，串行比例升高后需要更大的核。收益递减使同构方案在两端都不理想。Amdahl 定律的资源形式把核大小纳入模型，使这个权衡可以定量计算。

那么把串行部分专门交给一个胖核会怎样？

[5]视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

3 异构方案：一个胖核加若干瘦核

3.1 模型设定

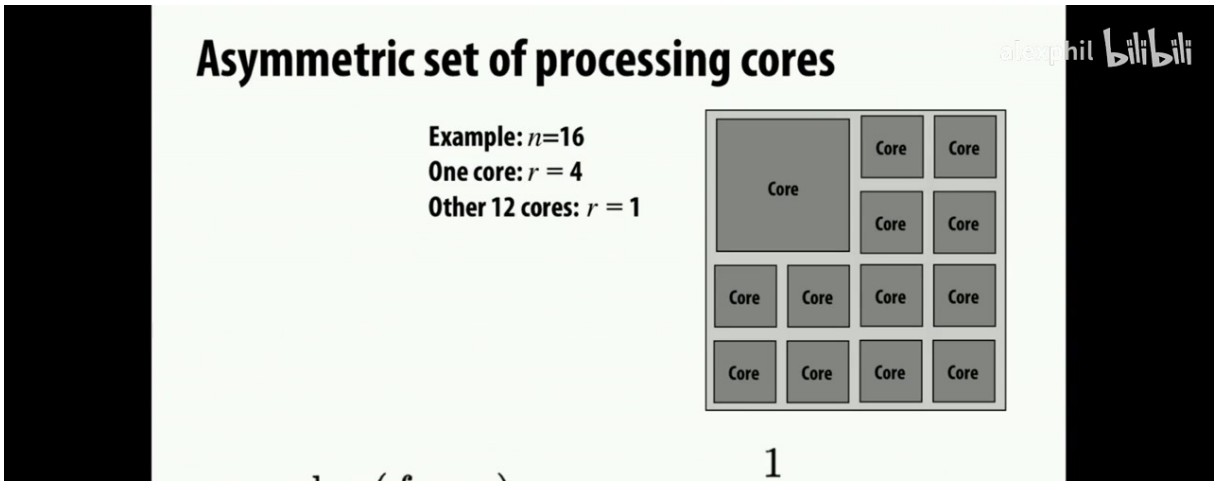


图 6: 非对称处理核集合：一个 $r = 4$ 的胖核加若干瘦核[6]

讲者提出一个很自然的问题：与其假设所有核都相同，不如做一个胖核加一批单位大小的瘦核。

三个参数是：可加速部分的比例、以基本小核为单位衡量的芯片总大小 N、以及分配给那个胖核的资源量 R。

基本想法是把串行部分映射到胖核上，把并行部分映射到瘦核上，从而从这个组合中获得收益。

$$S_{\text{hetero}} = \frac{1}{\dfrac{1-f}{\text{perf}(R)} + \dfrac{f}{\dfrac{N-R}{1}\cdot 1}}$$

- 第一项：串行部分在胖核上执行，其性能为 $\text{perf}(R)$；
- 第二项：并行部分分散到 $N - R$ 个瘦核上，每个性能为 1；
- 隐含假设是映射完美——串行代码全部跑在胖核上，并行代码全部跑在瘦核上。

[6] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

3.2 曲线更平缓

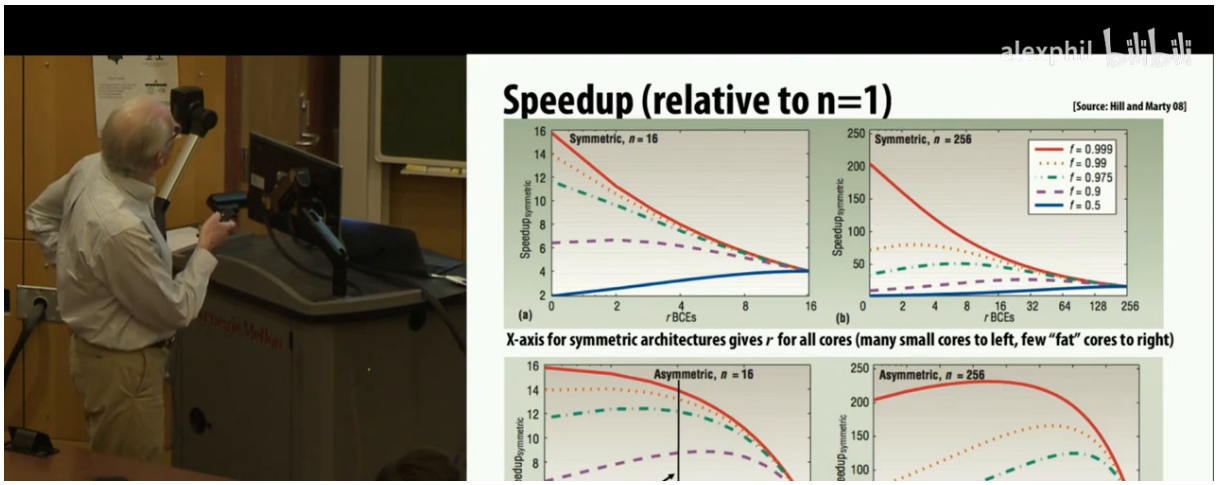


图 7: 非对称方案的加速比曲线（Hill 与 Marty 08）[7]

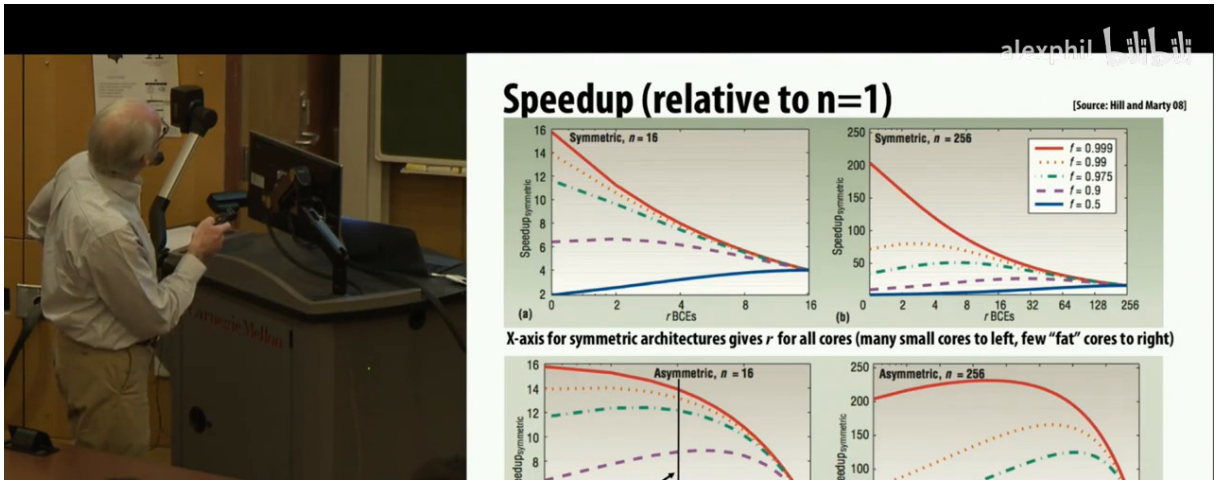


图 8: 非对称方案劣化更平缓，最优区间更宽[8]

讲者把非对称曲线与之前对称曲线放在一起比较。以 16 份总资源为例，若胖核占 0.25 的资源（即 $r = 4$），其余 12 份留给瘦核。结论很清楚：对称情形下曲线是陡峭下降，而非对称情形下劣化更平缓。

即便串行比例极低（0.01%），非对称方案也能在很宽的范围内接近理想并行。而更有意思的是那条 99% 并行的虚线，以及 2.5% 串行的情形——它们都" 撑得住"。

所有曲线仍会收敛到" 一个胖核占满整个芯片" 的点，但在此之前，非对称方案给出的收益明显更好。

讲者总结：把一部分面积专门用于顺序处理，然后把尽可能多的面积撒上一堆能做并行的小处理器，在这个模型下是个好主张。

最优的胖核大小可通过求导得到，其定性结论是：

$$R^* \propto \sqrt{1-f}$$

[7] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

- R^*：最优分配给胖核的资源量；
- f：可并行部分的比例；
- 串行比例越高（f 越小），胖核应分得越多资源。

这解释了为什么异构方案的最优区间比同构方案更宽：它多了一个可调参数。

异构的价值在于”让最坏情形也不至于太差”

同构方案的问题是最优核大小对串行比例极其敏感，一旦比例偏离假设就会大幅退化。异构方案牺牲一小块面积给胖核，换来的是对串行比例的鲁棒性：无论比例落在哪个量级，性能都不至于崩塌。这是用面积换稳健性的交易。

3.3 本章小结

异构方案把串行部分映射到胖核、并行部分映射到瘦核，曲线劣化更平缓、最优区间更宽。其价值在于对串行比例的鲁棒性，而不是更高的峰值。代价是一块面积被专用化。

但这个模型建立在若干乐观假设之上，现实约束会削弱它。

4 现实约束：为什么不能照模型定制

4.1 三条限制

讲者随即给出三条现实限制。

第一，这个模型假设我们能完美地把串行计算映射到胖核、把其余部分映射到其他核。这是乐观假设。

第二，很少有工作负载被刻画得如此清楚，可以围绕它设计整个系统。

第三，芯片开发成本达数百万美元，只有卖得足够多才能摊薄。而几乎没有哪个应用有足够的量来支撑一颗专用芯片。

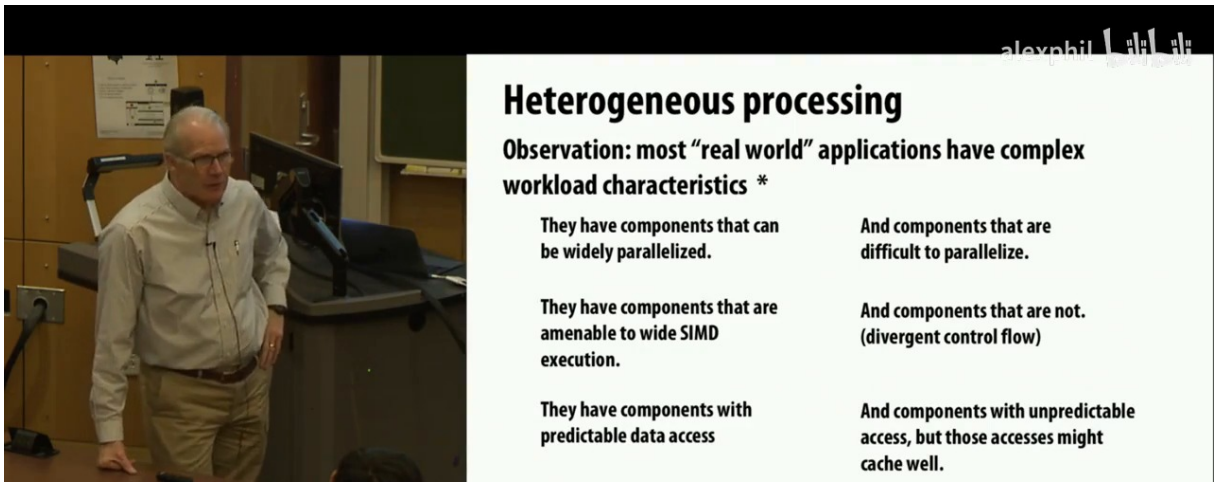


图 9：异构处理：真实应用的工作负载特征往往复杂多变[9]

[9] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

专用化与通用性是一对反向约束

专用化提升能效，但缩小了适用范围；通用性扩大适用范围，但牺牲能效。商用芯片必须服务未知负载，因此只能在两者之间取折中。这正是”异构”而非”专用”成为主流的原因：异构在保持通用性的同时获得部分专用化的收益。

他甚至指出手机也不是”一个负载”——它有各种各样不同的负载。因此把这些想法应用到实践中是个困难的工程权衡。

异构设计的真实约束是”通用性”

理想模型可以为一个已知串行比例定制芯片，但商用芯片必须服务未知负载。因此实际设计不追求针对某个比例的绝对最优，而追求在广泛负载范围内都不至于太差。这解释了为什么商用芯片的异构配置往往是”一个大核 + 若干中核 + 若干小核”这样的组合。

4.2 本章小结

异构方案把串行部分映射到胖核、并行部分映射到瘦核，结果是劣化更平缓、最优区间更宽，对串行比例更鲁棒。但现实限制有三：映射假设过于乐观、负载难以提前刻画、芯片成本要求足够销量。因此商用设计追求广泛负载下的稳健而非针对某点的最优。

下一章看真实系统如何落地这些想法。

5 真实系统里的异构

5.1 移动 SoC

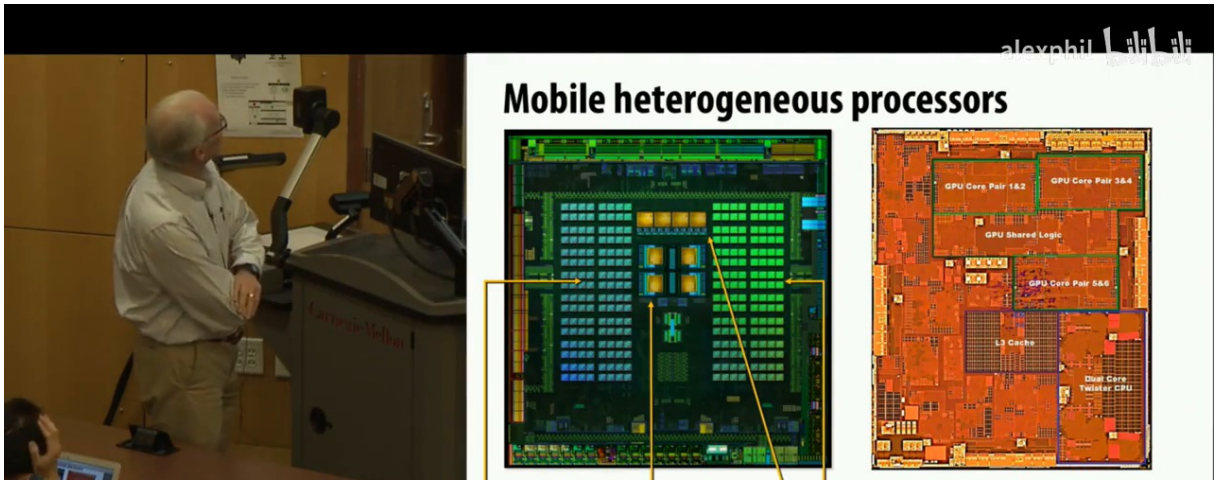


图 10: 移动端异构处理器：常规核、GPU 与共享逻辑[10]

讲者展示了一个移动处理器的布局，指出它已经在用异构核的想法。

常规核——也就是你登录进去跑线程的那些——只占整颗芯片的一部分，包含各自的一级与二级缓存。图中还有一块叫共享 LLC，即最后一级缓存（L3 的缓存控制器）。

[10] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

然后是一块 GPU。讲者特意说明：这不是 NVIDIA 那种可编程、能做非图形工作的 GPU，它们相当专门化于图形处理，可编程性与灵活性都不如 NVIDIA 的产品。但大多数笔记本（除非有独立显卡）就靠它。

他还指出，这个示例是四核版本，其他芯片会塞进更多核。但相当大一块面积专门给了图形控制。媒体引擎则负责 MPEG 视频解码之类的处理，这些是人们日常都在用的。

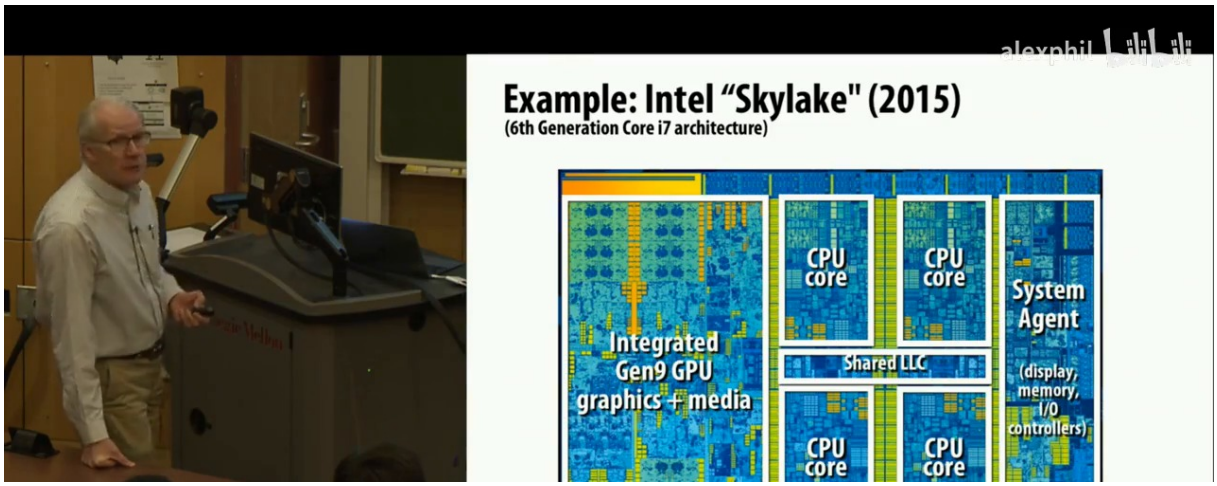


图 11: 实例：Intel Skylake（2015，第六代 Core i7）的布局[11]

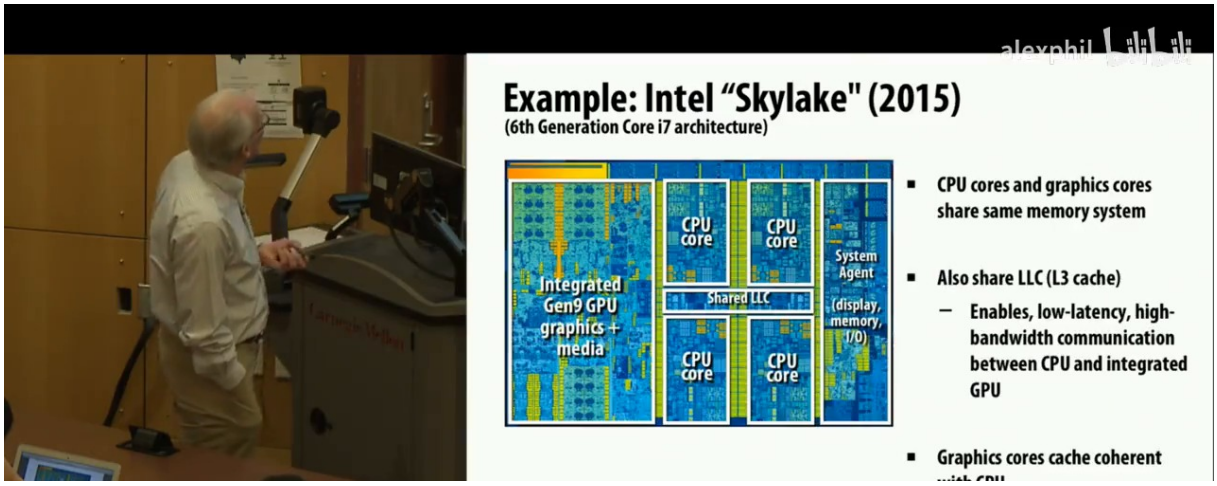


图 12: Skylake 中 CPU 核与图形核共享最后一级缓存[12]

5.2 片上集成的关键优势

讲者在这里点出一个性能上的关键差别：把所有东西放在一颗芯片上的好处之一是，GPU 也能共享这最后一级缓存。

于是它与内存系统之间有了非常高带宽的耦合。

这正好对应第 7 讲里你们在 Gates 机器上遇到的 NVIDIA GPU 的问题：你必须做 cudaMemcpy 来回搬运数据。那是个真正的性能杀手，因为它依赖外部总线的带宽，而不是芯片内部能提供的带宽。

[11] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

数据搬运的代价由互连带宽决定

外接加速器的算力再强，若每次调用都要把数据经外部总线搬进搬出，实际性能就会被搬运时间主导。片上集成把这条路径换成共享缓存，带宽高出若干倍，因此对”计算量相对数据量不大”的任务收益尤其明显。

5.3 再加一块独立 GPU

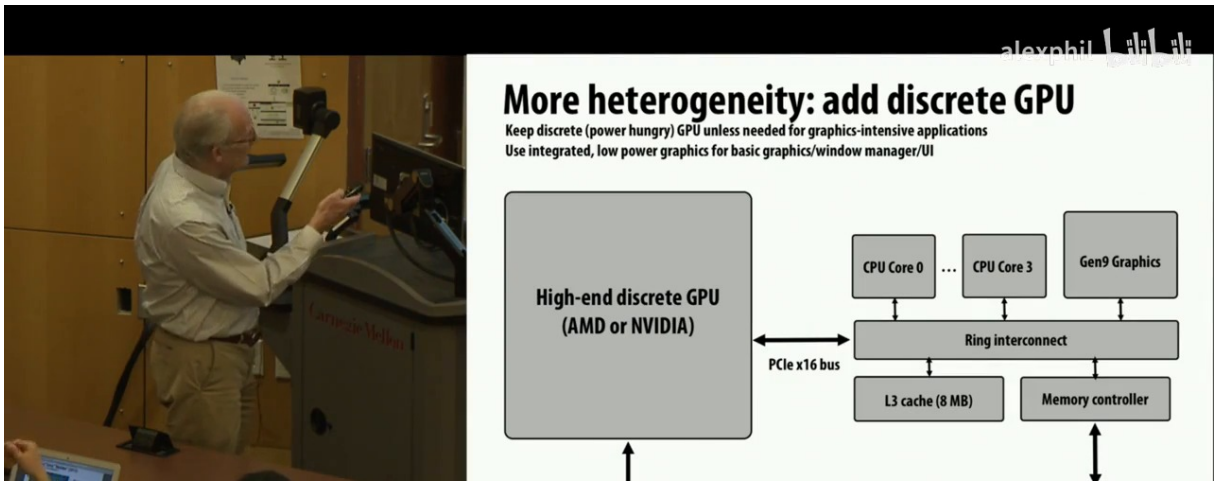


图 13: 更多异构：保留耗电的独立 GPU，仅在需要时启用[13]

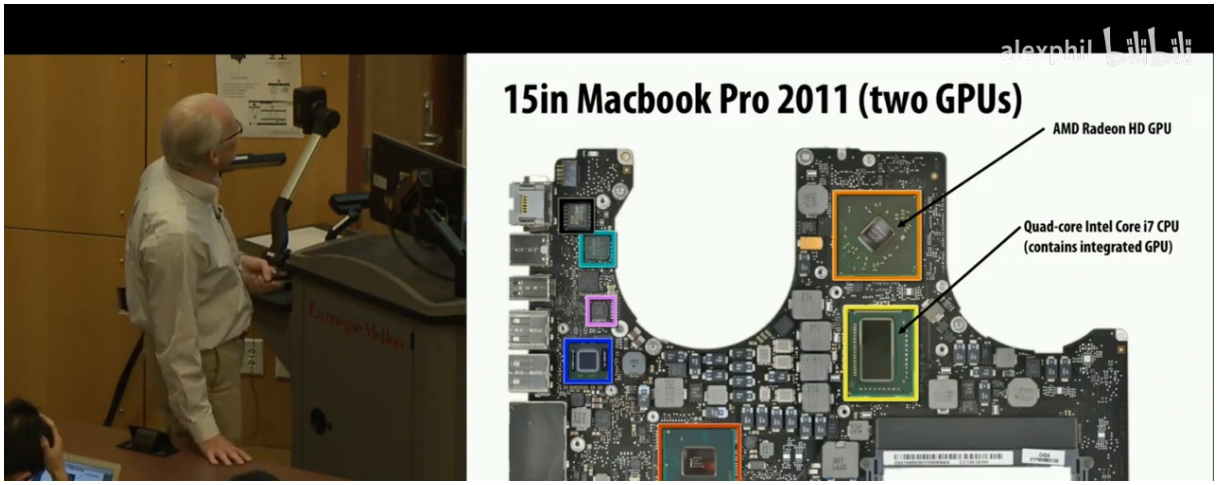


图 14: 15 寸 MacBook Pro 2011：四核 CPU 加两块 GPU[14]

讲者接着讨论更进一步的异构：加一块独立 GPU。策略是保留这块耗电的 GPU，但只在需要图形性能时才启用。

他用 2011 年 15 寸 MacBook Pro 举例：四核 Intel Core i7（内含集成 GPU）加一块 AMD Radeon 独立 GPU。因此典型配置是：一块轻度的集成 GPU 处理日常图形，加一块重量级 GPU 供需要时使用，中间由某种总线连接。

[13] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[14] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

多档加速器是功耗管理的具体手段

保留高性能但耗电的单元，仅在需要时启用，本质上是把”平均功耗”与”峰值性能”解耦。这与第 10 讲提到的手机芯片峰值 5 瓦、典型 100 到 200 毫瓦是同一逻辑：平时用低功耗单元，需要时短暂启用高功耗单元。

5.4 本章小结

真实系统普遍异构：移动 SoC 含常规核、专用 GPU 与媒体引擎；Skylake 让 CPU 核与图形核共享最后一级缓存；更高端配置再加一块独立 GPU 并按需启用。片上集成的关键优势是共享缓存带来的高带宽，避免了外接方案的数据搬运瓶颈。

6 超算与专用化

6.1 本章小结

真实系统普遍异构：移动 SoC 含常规核、专用 GPU 与媒体引擎；Skylake 让 CPU 核与图形核共享最后一级缓存；更高端配置再加一块独立 GPU 并按需启用。片上集成的关键优势是共享缓存带来的高带宽，避免了外接方案的数据搬运瓶颈。多档加速器本质上是把平均功耗与峰值性能解耦。

把视野放大到超算，异构还有另一层含义：基准如何定义性能。

7 超算排名与专用化

7.1 单一基准的问题

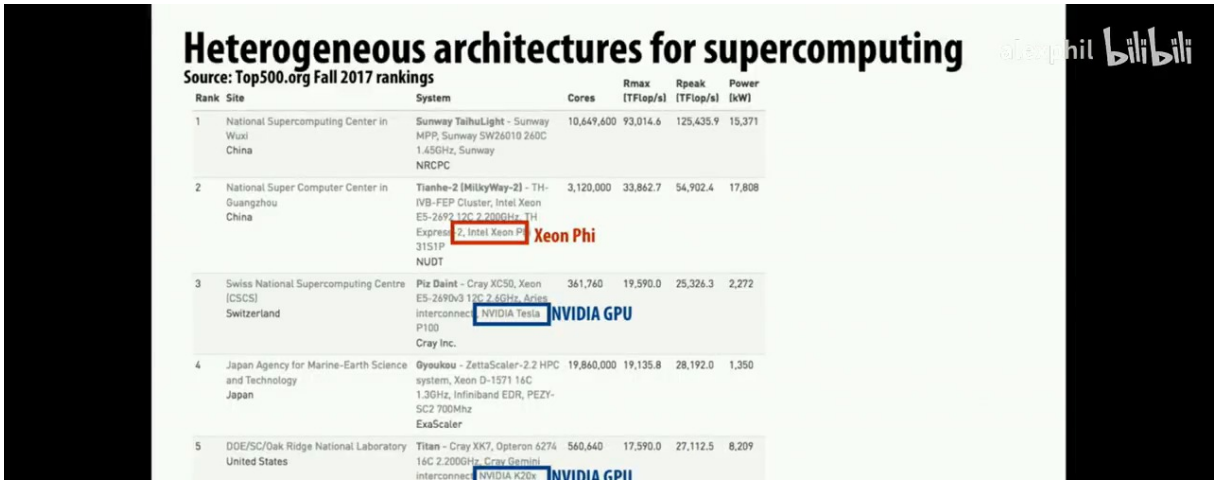


图 15: 超算的异构架构：Top500 排名数据[15]

[15] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

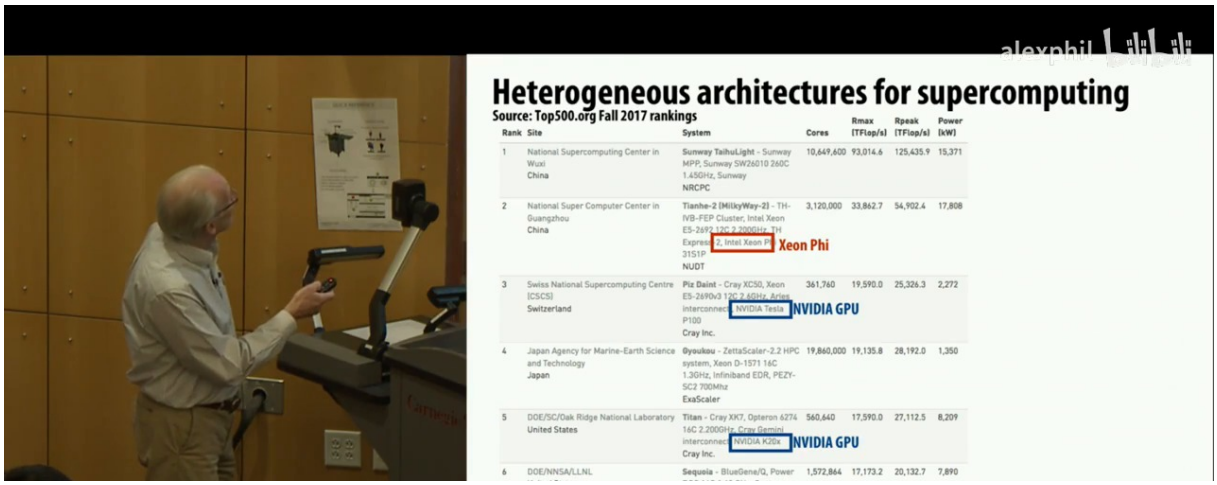


图 16: 以 Titan 为参照比较异构超算系统[16]

讲者对 Top500 排名提出了一个尖锐的批评，值得记录。

排名只跑一个基准：高斯消元，也就是用一个有 300 年历史的算法做矩阵求逆。这不能反映真实应用——即便是数值应用也会用更聪明的算法，通常处理稀疏矩阵，而这个是稠密矩阵。

因此这个基准把大量权重放在原始浮点能力上，而很少反映可编程性、互连网络质量这些实际决定应用性能的因素。

讲者还补充了一个现实原因：做这件事的人会花整整一年甚至更久、用整个团队去调优这一个基准。而如果调十个基准，工作量就是十倍，所以做基准的人本身也不愿意扩展。

另一个原因是要锁定整台机器：这种规模的机器通常不会只跑一个应用，而是被分成几十份。但做这些测试需要完全不受干扰的访问权。因此部分原因是后勤问题。

单一基准会系统性偏向某一类架构

只用稠密浮点基准排名，会高估那些为稠密矩阵优化的架构，低估那些强在可编程性、互连或数据局部性的架构。因此排名可作为参考，但不能当作”通用性能”的度量。评价系统必须结合目标应用的通信与访存特征。

[16] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

7.2 专用化：走向硬件

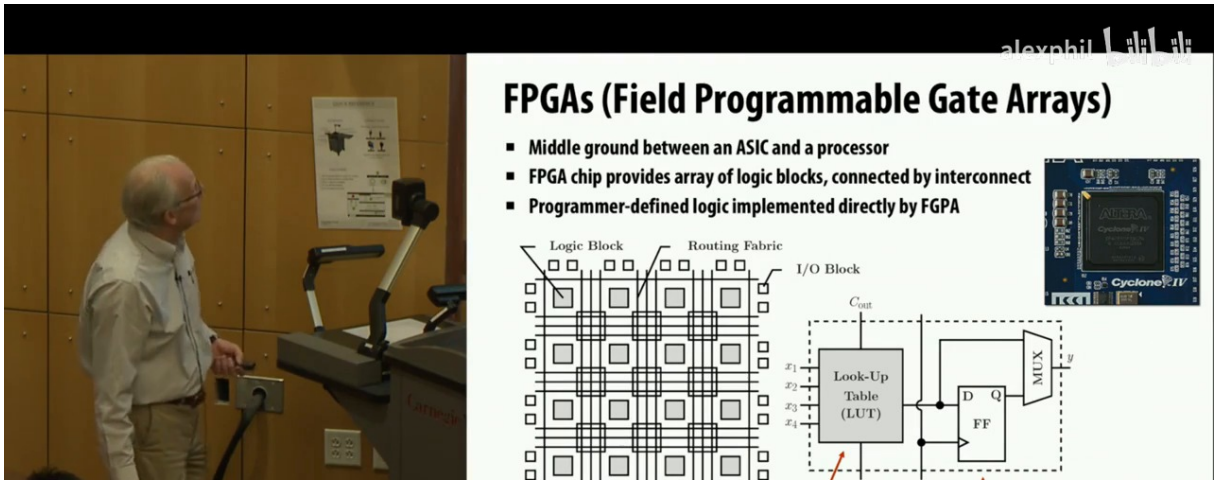


图 17: FPGA：介于 ASIC 与处理器之间的中间地带[17]

讲者最后讨论另一个维度：从纯软件走向更接近硬件，甚至直接做硬件设计。

他提到 FPGA 处在一个中间地带：FPGA 芯片提供一片可编程逻辑阵列，因此既能获得接近硬件的能效，又保留了一定的可重配置性，不必像 ASIC 那样一次性流片。

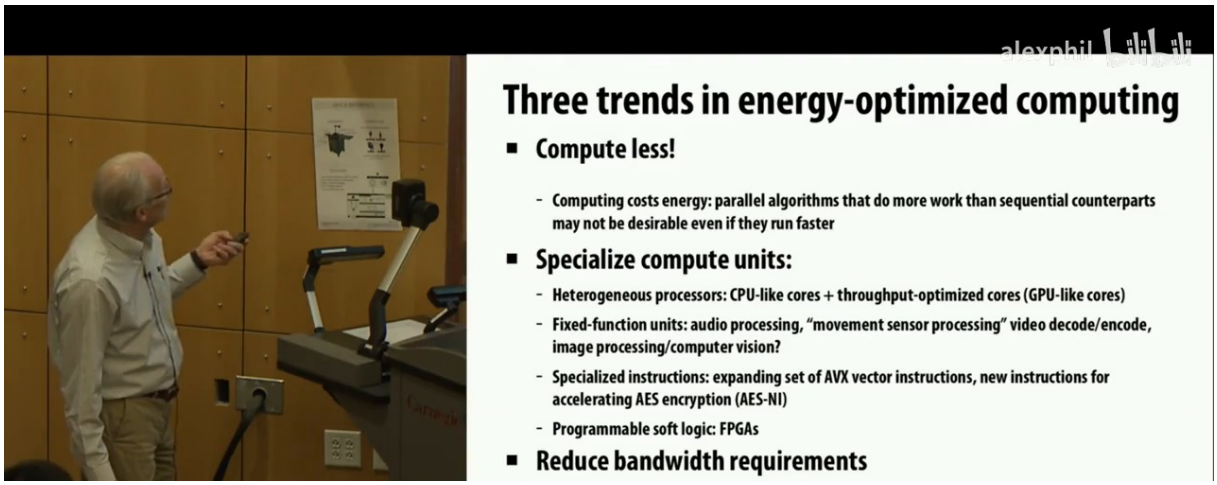


图 18: 能效优化的三条趋势：减少计算量、并行算法、专用化[18]

7.3 本章小结

Top500 只用高斯消元一个基准，系统性地偏向稠密浮点能力，忽略可编程性与互连质量，且调优成本与后勤约束使这一状况难以改变。专用化是另一条路径，FPGA 提供了介于 ASIC 与通用处理器之间的中间地带。

[17] 视频画面时间区间：01:10:00–01:10:15。若为裁剪图，时间区间仍对应原视频画面。

[18] 视频画面时间区间：01:19:00–01:19:15。若为裁剪图，时间区间仍对应原视频画面。

8 总结与延伸

讲者在结尾说这是一个有大量工作在进行、并且随时间不断变化的领域，是整个系统中非常活跃的一部分。今天的系统原理与十年前差别很大，部分原因正是这些功耗问题。

把全讲串起来，这是一条从” 功耗约束” 到” 异构设计” 再到” 真实系统” 的路径。功耗是从手机到超算的共同主导约束，它把性能问题重新定义为每瓦性能问题；同时现有硬件的相当部分在多数程序中被闲置。在固定面积模型下，胖核与瘦核的取舍由串行比例决定：几乎全并行时小核最优，串行比例升高后需要更大的核，而收益递减使同构方案在两端都不理想。异构方案把串行部分映射到胖核、并行部分映射到瘦核，结果是劣化更平缓、对串行比例更鲁棒，代价是一块面积被专用化。但现实限制有三：映射假设过于乐观、负载难以提前刻画、芯片成本要求足够销量。真实系统普遍异构：移动 SoC 含常规核、专用 GPU 与媒体引擎；Skylake 让 CPU 核与图形核共享最后一级缓存；更高端配置再加独立 GPU 并按需启用，片上集成的关键优势是避免外接方案的数据搬运瓶颈。最后，Top500 的单一基准会系统性偏向稠密浮点能力，而专用化（如 FPGA）是另一条提升能效的路径。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 异构的价值不是” 更多算力”，而是” 对负载变化的鲁棒性”。同构设计在已知负载下可以做到最优，但对负载比例极其敏感。异构用一小块专用面积换取跨负载的稳定表现，这是稳健性交易而非峰值交易。
- 收益递减是整套论证的关键前提。若单核性能能随面积线性增长，把所有面积给一个核就是最优，异构便没有意义。因此评估异构是否值得，首先要问单核性能的实际面积缩放曲线。
- 片上与外接的差别主要是带宽而非算力。外接加速器算力可能更强，但数据搬运受外部总线限制。因此” 计算量相对数据量越大” 的任务越适合外接，” 数据量大而计算相对轻” 的任务越适合片上。
- 基准的选择本身就是一种架构主张。只用稠密浮点基准排名，等于宣布这类能力最重要。因此读排名时必须同时问：这个基准与我的应用有多相似。

这堂课的边界在于它是导论性讨论：讲者给出的是权衡模型与真实系统的观察，未深入具体加速器的编程模型与调度机制。

两个可以直接从本讲数据提出的下一步问题：其一，异构方案依赖” 串行部分能映射到胖核” 这一假设，那么当串行部分本身无法被识别或分离时，异构的收益还剩多少？其二，片上共享缓存避免了数据搬运，但也意味着 CPU 与 GPU 竞争同一份缓存容量，这个竞争在什么工作负载下会抵消共享带来的收益？